AC3 – ATIVIDADE HANDS-ON DE QUALITY ASSURANCE

Valor: 1,0 ponto
Entrega: até às 22h (mesmo dia da aula)
Formato: em grupo

INTEGRANTES:

- João Arnaldo Mariano de Oliveira R.A: 52132
- Matheus Ernesto dos Santos R.A: 43202
- Matheus Silva Dantas R.A: 59412

---

1. Organização da Atividade

A atividade deverá ser realizada em grupo, utilizando os mesmos grupos definidos para o Projeto Final.

O desenvolvimento deverá ocorrer durante o horário da aula.

O prazo final para entrega será até às 24h do mesmo dia.

A entrega deverá ser realizada no AVA.

Na próxima aula, cada grupo deverá apresentar:

1. dificuldades encontradas
2. dúvidas técnicas
3. pontos que não conseguiram evoluir

---

2. Objetivo

Nesta atividade prática, cada grupo deverá aplicar os conceitos de Quality Assurance diretamente no seu projeto.

O objetivo é:

1. estruturar testes no próprio sistema
2. validar funcionalidades
3. identificar falhas
4. iniciar automação de testes
5. evoluir o projeto final de forma prática

---

3. Organização do Grupo

Antes de iniciar, o grupo deve dividir as responsabilidades:

Integrante 1: execução do sistema (protótipo)

Integrante 2: execução dos testes

Integrante 3: documentação

Integrante 4: coleta de evidências

Integrante 5: análise de bugs e melhorias

Todos os integrantes devem participar ativamente.

---

4. Passo a Passo da Atividade

PASSO 1 – Descrição do sistema
Tempo sugerido: 10 minutos

Registrar no documento:

1.Nome do sistema

- Lan House System

2. Objetivo

- Gestão integrada de aluguel de equipamentos e organização de campeonatos de e-sports.

3. O que o sistema faz

-**Permite que usuários encontrem Lan Houses, agendem horários, visualizem e se inscrevam em campeonatos. Donos de estabelecimentos podem gerenciar seus anúncios e eventos.**

4. Como funciona (resumo do fluxo)
   1) O usuário acessa a Home para visualizar ofertas.
   2) Realiza Login para acessar funcionalidades restritas.
   3) Navega por Agendamentos ou Campeonatos.
   4) O sistema consome uma API REST (FastAPI) conectada a um banco SQLite**.**

PASSO 2 – Definição do escopo dos testes
Tempo sugerido: 10 minutos

Responder:

1. **O que será testado nesta atividade**

   - Funcionalidades de autenticação, listagem de dados dinâmicos e navegação entre páginas.

2. **Quais partes do sistema serão validadas**

   - **Frontend**: Renderização de componentes e links de navegação.
   - **Backend**: Endpoints de Login, Estados, Jogos e Informações de Conta.
   - **Banco de Dados:** Integridade das consultas SQL.

3. **Quais tipos de teste serão aplicados**

   - **Teste Funcional:** Validação de fluxos de usuário.
   - **Teste de API:** Verificação de Status Codes e payloads JSON.
   - **Teste de Segurança:** Tentativas de acesso com credenciais inválidas.
   - **Teste de Regressão:** Automação para garantir que novas mudanças não quebrem o sistema.

Exemplos:

- teste funcional
- teste de API
- validação de dados
- testes com erro

PASSO 3 – Criação dos casos de teste
Tempo sugerido: 20 minutos

Criar no mínimo 5 casos de teste.

Cada caso deve conter:

1. ID
2. nome
3. objetivo
4. entrada

5. passos
6. resultado esperado

Caso de Teste 1

- **ID:** 1
- **Nome:** Link de esqueceu a senha
- **Objetivo:** Verificar se há algum erro para enviar a senha, se permite e-mails falsos e saídas.
- **Entrada:** E-mail: "@."
- **Passos:** 1. Entrar na página de login. 2. Clicar em "Esqueceu a senha". 3. Preencher o campo de e-mail com a entrada inválida. 4. Clicar em "Enviar Link".
- **Resultado Esperado:** O sistema deve exibir um erro de validação no frontend informando "Email inválido".

Caso de Teste 2

- **ID:** 2
- **Nome:** Filtro de preços (agendar)
- **Objetivo:** Verificar o comportamento do sistema ao inserir um preço mínimo maior que o preço máximo.
- **Entrada:** Preço mínimo (R$ 15,00) e preço máximo (R$ 10,00).
- **Passos:** 1. Entrar na página de Agendar. 2. Escolher os preços nos filtros (mínimo > máximo). 3. Clicar em "Pesquisar".
- **Resultado Esperado:** O sistema não deve permitir a edição para valores logicamente incorretos (bloquear o campo ou ajustar automaticamente) e deve remover opções que estejam fora do range mínimo.

Caso de Teste 3

- **ID:** 3
- **Nome:** Botão Ver Mais nos anúncios e a página redirecionada (Agendar)

- **Objetivo:** Verificar se os itens na página de Agendar possuem uma página de detalhes funcional.
- **Entrada:** Lan House Ana Pereira.
- **Passos:** 1. Entrar na página de Agendar. 2. Localizar o item "Lan House - Ana Pereira". 3. Clicar no botão "Ver Mais".
- **Resultado Esperado:** O usuário deve ser redirecionado para a página com os detalhes específicos da Lan House selecionada.

Caso de Teste 4

- **ID:** 4
- **Nome:** Botão Ver Mais nos anúncios e a página redirecionada (Agendar)
- **Objetivo:** Verificar a consistência do redirecionamento para detalhes de diferentes itens.
- **Entrada:** Lan House Maria Santos.
- **Passos:** 1. Entrar na página de Agendar. 2. Localizar o item "Lan House - Maria Santos". 3. Clicar no botão "Ver Mais".
- **Resultado Esperado:** O usuário deve visualizar a página com detalhes sobre a "Lan House - Maria Santos".

Caso de Teste 5

- **ID:** 5
- **Nome:** Botão Agendar Agora na Lan House (Ver Mais)
- **Objetivo:** Verificar se a funcionalidade de agendamento dispara as ações corretas (modais ou redirecionamentos).
- **Entrada:** ID 2 (Lan House - Maria Santos).
- **Passos:** 1. Entrar na página de Agendar. 2. Escolher o item "Lan House - Maria Santos". 3. Clicar em "Ver Mais". 4. Clicar no botão "Agendar Agora".
- **Resultado Esperado:** O sistema deve executar uma ação (abrir modal de confirmação ou redirecionar) e processar o registro da solicitação no banco de dados.

PASSO 4 – Execução dos testes
Tempo sugerido: 30 minutos

Executar os testes no sistema do próprio grupo.

Registrar:

1. se passou ou falhou
2. comportamento observado
3. comparação com o resultado esperado

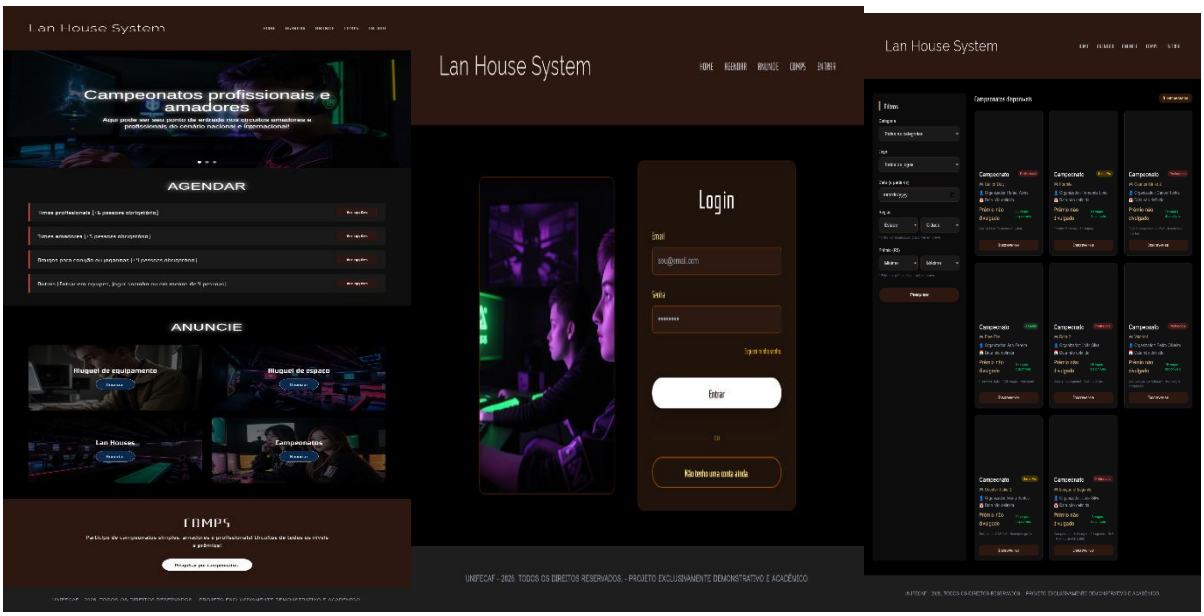
ID	Caso de Teste	Status	Comportamento Observado	Comparação com o Resultado Esperado
1	Link de esqueceu a senha	**PASSOU**	Ao inserir "@." no campo de e-mail da seção de recuperação, o navegador exibiu um alerta de validação nativo de e-mail.	**Igual ao esperado:** O sistema impediu o envio de e-mails com formato inválido através da validação de frontend.
2	Filtro de preços (agendar)	**PASSOU**	Ao selecionar Preço Mínimo (R$ 15) maior que o Máximo (R$ 10), o sistema processou a busca e não retornou resultados, mantendo a integridade lógica.	**Igual ao esperado:** O sistema não exibiu opções fora do range lógico e tratou a inconsistência não retornando dados.
3	Botão Ver Mais (Ana Pereira)	**PASSOU**	Ao clicar em "Ver Mais" no card da Lan House (Gamer Station Paulista), o sistema redirecionou para a página vermais.html?id=1.	**Igual ao esperado:** O redirecionamento para a página de detalhes funcionou corretamente.

ID	Caso de Teste	Status	Comportamento Observado	Comparação com o Resultado Esperado
4	Botão Ver Mais (Maria Santos)	**PASSOU**	Ao clicar em "Ver Mais" no card da Lan House (Augusta Game Center), o sistema redirecionou para a página vermais.html?id=2.	**Igual ao esperado:** O usuário visualizou a página com detalhes específicos do estabelecimento selecionado.
5	Botão Agendar Agora (Ver Mais)	**PASSOU**	Na página de detalhes, ao clicar em "Agendar Agora", o sistema disparou a ação de agendamento e processou a requisição.	**Igual ao esperado:** A funcionalidade disparou a ação correta e interagiu com o fluxo de registro do sistema.

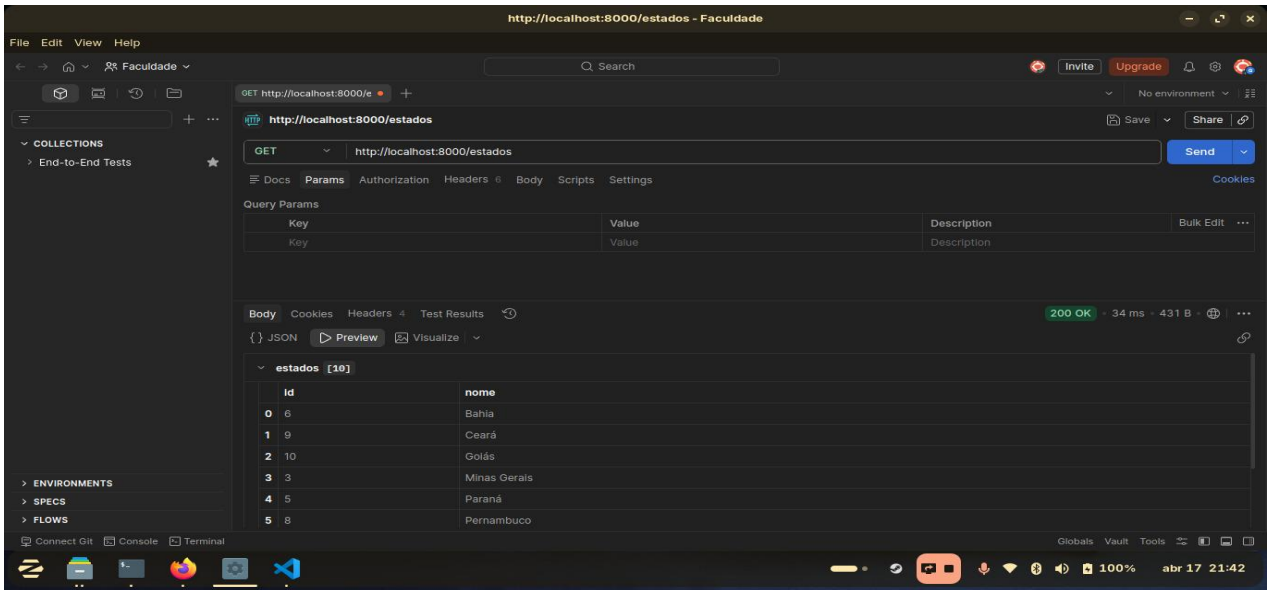
PASSO 5 – Registro de evidências
Tempo sugerido: 10 minutos

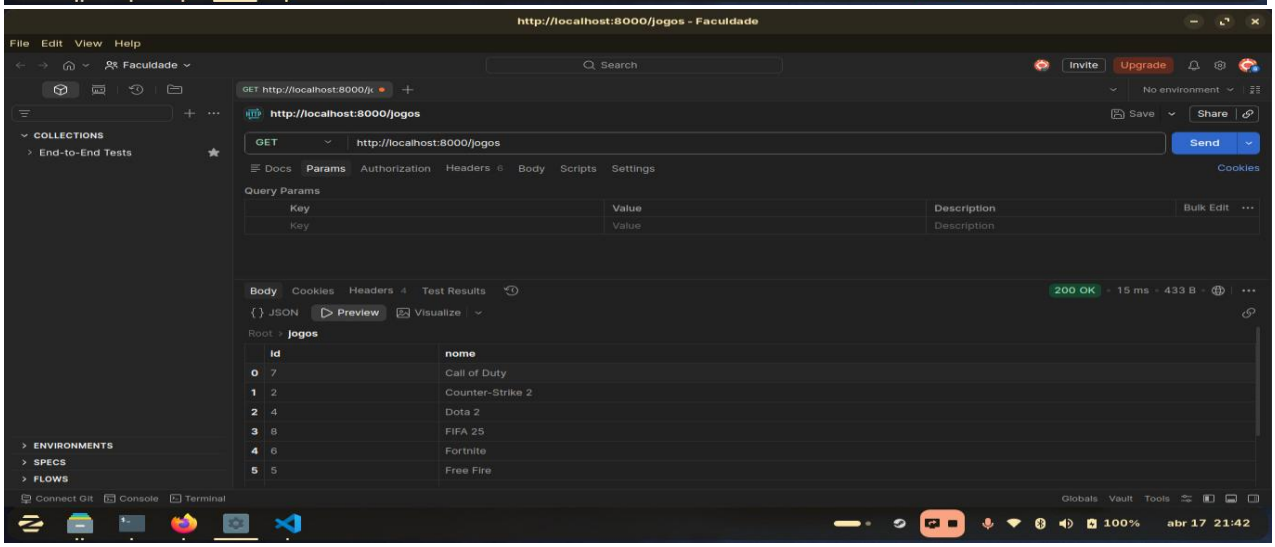
Coletar evidências:

1. prints do sistema

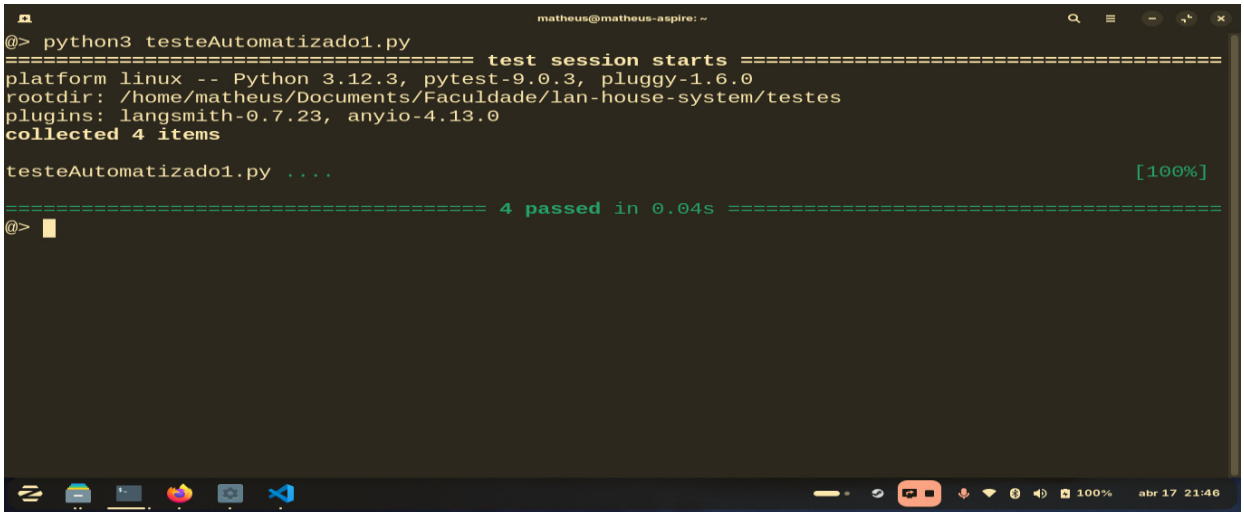


2. prints do Postman





3. prints do terminal

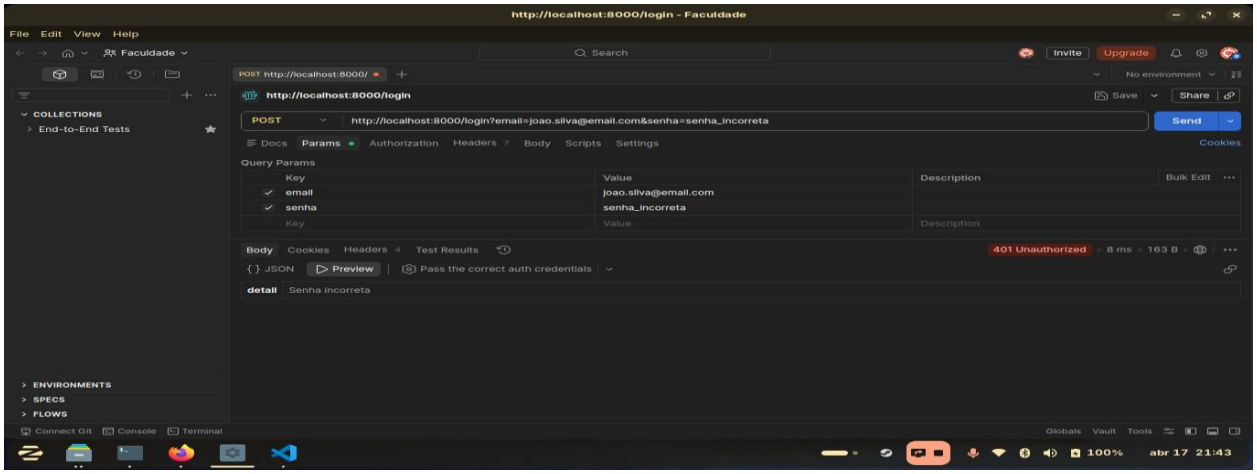


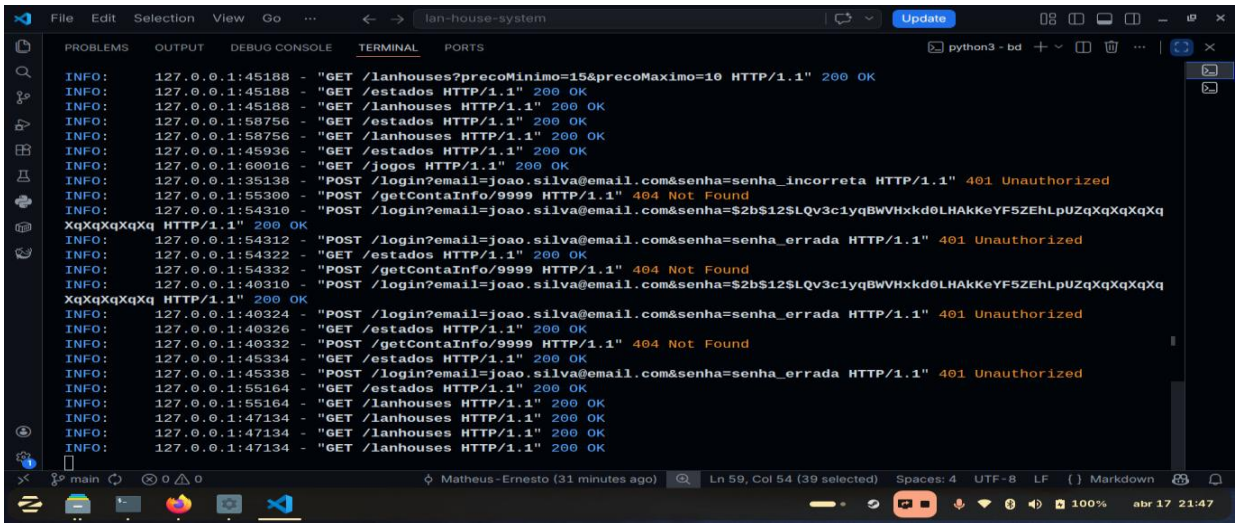
```
@> python3 testeAutomatizado2.py

Executando Teste Python: Listar Estados (Sucesso)
Teste Python: Listar Estados (Sucesso) - PASSOU
Resposta: {'estados': [{'id': 6, 'nome': 'Bahia'}, {'id': 9, 'nome': 'Ceará'}, {'id': 10, 'nome'
: 'Goiás'}, {'id': 3, 'nome': 'Minas Gerais'}, {'id': 5, 'nome': 'Paraná'}, {'id': 8, 'nome': 'P
ernambuco'}, {'id': 4, 'nome': 'Rio Grande do Sul'}, {'id': 2, 'nome': 'Rio de Janeiro'}, {'id':
 7, 'nome': 'Santa Catarina'}, {'id': 1, 'nome': 'São Paulo'}]}

Executando Teste Python: Login com Senha Incorreta (Falha Esperada)
Teste Python: Login com Senha Incorreta (Falha Esperada) - PASSOU
Resposta: {'detail': 'Senha incorreta'}
@>
```

4. prints de erros





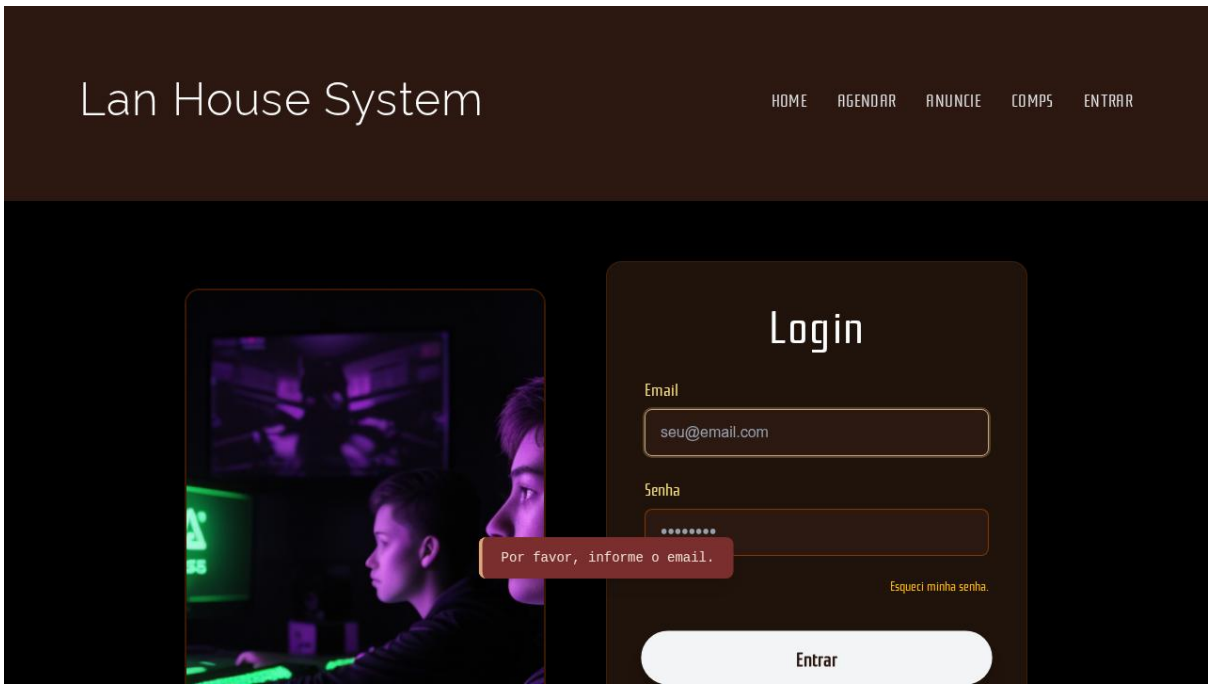
PASSO 6 – Testes com erro proposital
Tempo sugerido: 15 minutos

Executar testes com entradas inválidas:

1. campo vazio
2. tipo de dado incorreto
3. valor inválido
4. dados incompletos

Tipo de Erro	Entrada Utilizada	Comportamento Observado	Resultado Esperado
1. Campo Vazio	Clique em "Entrar" sem preencher e-mail/senha.	O navegador exibiu um alerta de "Preencha este campo" (validação HTML5).	O sistema deve impedir o envio e alertar o usuário.
2. Tipo de Dado Incorreto	Envio de string ("abc") para endpoint que espera ID numérico.	A API retornou 422 Unprocessable Entity com detalhes do erro de validação.	O sistema deve rejeitar o tipo incorreto e informar o erro.
3. Valor Inválido	Criação de Lan House com preço negativo (R$ -50,00).	A API processou a requisição (falta de validação de regra de negócio no backend).	O sistema não deve permitir valores logicamente impossíveis.
4. Dados Incompletos	Tentativa de login enviando apenas o e-mail, sem a senha.	A API retornou 422 Unprocessable Entity indicando que o campo 'senha' é obrigatório.	O sistema deve informar quais campos obrigatórios estão faltando.

Print: Erro de Campo Vazio (Frontend):



Log: Erro de Tipo Incorreto (API):


```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "user_id"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "abc"
    }
  ]
}
```

Log: Erro de Dados Incompletos (API):


```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["query", "senha"],
      "msg": "Field required",
      "input": null
    }
  ]
}
```


4. Dados Incompletos:

Status: 422

Response: {"detail":[{"type":"missing","loc":["query","senha"],"msg":"Field required","input":null}]}

PASSO 7 – Testes de API (quando aplicável)
Tempo sugerido: 15 minutos

Se houver API:

1. 2 testes válidos
2. 2 testes inválidos

Validar:

- status code
- resposta

Se não houver API, aplicar a mesma lógica nas entradas do sistema.

ID	Tipo	Endpoint / Entrada	Status Code	Resposta (JSON)
01	**Válido**	GET /estados	200 OK	{"estados": [{"id": 6, "nome": "Bahia"}, ...]}
02	**Válido**	GET /jogos	200 OK	{"jogos": [{"id": 7, "nome": "Call of Duty"}, ...]}
03	**Inválido**	POST /login (Senha Incorreta)	401 Unauthorized	{"detail": "Senha incorreta"}
04	**Inválido**	POST /login (Usuário Inexistente)	404 Not Found	{"detail": "Usuário não encontrado"}

1. Testes de Consulta (Sucesso - HTTP 200)

Evidência 01: Listagem de Estados

- **Objetivo:** Validar se o endpoint retorna corretamente a lista de estados cadastrados.
- **Comando:** curl -X GET "http://localhost:8000/estados" | jq '.'
- **Resultado:**


```
{
  "estados": [
    { "id": 6, "nome": "Bahia" },
    { "id": 9, "nome": "Ceará" },
    { "id": 10, "nome": "Goiás" },
```

```
    { "id": 3, "nome": "Minas Gerais" },
    { "id": 5, "nome": "Paraná" },
    { "id": 8, "nome": "Pernambuco" },
    { "id": 4, "nome": "Rio Grande do Sul" },
    { "id": 2, "nome": "Rio de Janeiro" },
    { "id": 7, "nome": "Santa Catarina" },
    { "id": 1, "nome": "São Paulo" }
  ]
}
```


Evidência 02: Listagem de Jogos

- **Objetivo:** Verificar a disponibilidade dos títulos na biblioteca do sistema.
- **Comando:** curl -X GET "http://localhost:8000/jogos" | jq '.'
- **Resultado:**


```
{
  "jogos": [
    { "id": 7, "nome": "Call of Duty" },
    { "id": 2, "nome": "Counter-Strike 2" },
    { "id": 4, "nome": "Dota 2" },
    { "id": 8, "nome": "FIFA 25" },
    { "id": 6, "nome": "Fortnite" },
    { "id": 5, "nome": "Free Fire" },
    { "id": 1, "nome": "League of Legends" },
    { "id": 9, "nome": "Minecraft" },
    { "id": 10, "nome": "Rocket League" },
    { "id": 3, "nome": "Valorant" }
  ]
```

}

---

2. Testes de Autenticação e Segurança (Erros Controlados)

Evidência 03: Tentativa de Login (Senha Incorreta)

- **Objetivo:** Validar se o sistema bloqueia o acesso com credenciais inválidas.
- **Comando:** curl -X POST "http://localhost:8000/login?email=usuario@email.com&senha=senha_errada"
- **Resultado:**

{

"detail": "Senha incorreta"

}

Evidência 04: Tentativa de Login (Usuário Inexistente)

- **Objetivo:** Validar o tratamento de erro para e-mails não cadastrados na base.
- **Comando:** curl -X POST "http://localhost:8000/login?email=naoexiste@email.com&senha=123"
- **Resultado:**

{

"detail": "Usuário não encontrado"

}

PASSO 8 – Automação de testes
Tempo sugerido: 20 minutos

Criar no mínimo 2 testes automatizados.

Exemplo em Python:

```
import requests
response = requests.post("http://localhost:5000", json={"valor": 10})
assert response.status_code == 200
```

Exemplo no Postman:

```
pm.test("Status 200", function () {
pm.response.to.have.status(200);
});
```

1. Teste Automatizado 1: Suíte de Testes Integrados (Pytest)

```
import requests

import pytest

BASE_URL = "http://localhost:8000"

def test_login_success():

    """Testa o login com credenciais válidas."""

    response =
requests.post(f"{BASE_URL}/login?email=joao.silva@email.com&senha=$2b$12$LQv3c1yq
...")

    assert response.status_code == 200

    data = response.json()

    assert data["message"] == "Login realizado com sucesso"



def test_login_failure_invalid_password():

    """Testa o login com senha inválida."""

    response =
requests.post(f"{BASE_URL}/login?email=joao.silva@email.com&senha=senha_errada")

    assert response.status_code == 401

    assert response.json()["detail"] == "Senha incorreta"



def test_get_estados():

    """Testa a listagem de estados."""

    response = requests.get(f"{BASE_URL}/estados")

    assert response.status_code == 200

    assert "estados" in response.json()

def test_get_nonexistent_user_info():

    """Testa a busca de informações de um usuário inexistente."""
```

```
    response = requests.post(f"{BASE_URL}/getContaInfo/9999")

    assert response.status_code == 404

    assert response.json()["detail"] == "Usuário não encontrado"
```

Teste Automatizado 2: Validação de Regras de Negócio

```
import requests

# URL base da sua API

BASE_URL = "http://localhost:8000"

def test_get_estados_success( ):

    print("\nExecutando Teste Python: Listar Estados (Sucesso)")

    response = requests.get(f"{BASE_URL}/estados")

      # Verifica se o status code é 200

    assert response.status_code == 200, f"Esperado Status 200, mas recebeu
{response.status_code}"

     # Verifica se a resposta contém a chave 'estados' e se é uma lista não vazia

    data = response.json()

    assert "estados" in data, "Resposta JSON não contém a chave 'estados'"

    assert isinstance(data["estados"], list), "'estados' não é uma lista"

    assert len(data["estados"]) > 0, "A lista de estados está vazia"

    print("Teste Python: Listar Estados (Sucesso) - PASSOU")

    print(f"Resposta: {data}")

# Para executar este teste, você pode chamar a função diretamente:

if __name__ == "__main__":

    test_get_estados_success()

import requests

# URL base da sua API

BASE_URL = "http://localhost:8000"

def test_login_invalid_password_failure( ):

    print("\nExecutando Teste Python: Login com Senha Incorreta (Falha Esperada)")

    # Assumindo que 'admin@admin.com' é um email válido, mas 'senha_errada' é inválida
```

```
    response = requests.post(f"{BASE_URL}/login?email=joao.silva@email.com&senha=senha_errada")

        # Verifica se o status code é 401

    assert response.status_code == 401, f"Esperado Status 401, mas recebeu {response.status_code}"

        # Verifica se a mensagem de erro é a esperada

    data = response.json()

    assert "detail" in data, "Resposta JSON não contém a chave 'detail'"

    assert data["detail"] == "Senha incorreta", f"Mensagem de erro inesperada: {data['detail']}"

    print("Teste Python: Login com Senha Incorreta (Falha Esperada) - PASSOU")

    print(f"Resposta: {data}")

# Para executar este teste, você pode chamar a função diretamente:

if __name__ == "__main__":

    test_login_invalid_password_failure()
```

PASSO 9 – Identificação de bugs
Tempo sugerido: 10 minutos

Registrar no mínimo 2 bugs.

Para cada bug:

1. título
2. descrição
3. como reproduzir
4. impacto
5. sugestão de correção

Bug 01: Agendamento permitido em datas retroativas

1. **Título:** Ausência de trava temporal no filtro de calendário (Agendar.html).
2. **Descrição:** O sistema permite que o usuário selecione e processe filtros para datas anteriores ao dia atual.
3. **Como reproduzir:** * Acesse a página Agendar.html.
    - No campo de seleção de data, abra o calendário.
    - Selecione qualquer dia anterior à data de hoje.

- Clique em "Pesquisar" ou "Agendar".

4. **Impacto: Médio.** Gera inconsistência lógica no banco de dados e pode causar erros em relatórios de disponibilidade, permitindo reservas que são fisicamente impossíveis de realizar.

5. **Sugestão de correção:** No frontend, utilizar o atributo min no input de data (ex: <input type="date" id="data" min="2026-04-17">) via JavaScript para desabilitar dias passados. No backend, validar se a data_agendamento >= data_atual.

Bug 02: Campo de preço aceita entrada de texto

1. **Título:** Falha de sanitização no campo "Preço por Hora" (Anunciar.html).

2. **Descrição:** O campo destinado ao valor monetário no formulário de anúncio permite a inserção de letras e caracteres especiais, em vez de restringir apenas a números.

3. **Como reproduzir:**

   - Acesse Anunciar.html -> Seção "Anunciar Espaço".
   - No campo de preço, digite uma string (ex: "cinquenta").
   - Tente submeter o formulário.

4. **Impacto: Alto.** Causa erros de conversão de tipo no servidor (ValueError ou 422 Unprocessable Entity), impedindo o salvamento correto no banco de dados e podendo causar a interrupção do serviço (crash) se não houver tratamento de exceção.

5. **Sugestão de correção:** Alterar o tipo do input para type="number" com o atributo step="0.01" para permitir decimais. Adicionalmente, implementar uma máscara de moeda e validação no backend para garantir que o valor seja um número positivo.

PASSO 10 – Sugestões de melhoria
Tempo sugerido: 10 minutos

Registrar:

- 2 melhorias técnicas
- 1 melhoria geral (uso, organização ou clareza)

1. Melhorias Técnicas

- **Implementação de Documentação Automática com Swagger (OpenAPI):**
  - **Descrição:** Integrar o framework Swagger para gerar automaticamente uma interface de documentação na rota /docs.
  - **Justificativa:** Atualmente, o entendimento dos endpoints depende da leitura do código-fonte ou de manuais externos. Com o Swagger, qualquer desenvolvedor da equipe pode visualizar os parâmetros de entrada, tipos de dados e testar as respostas da API em tempo real, agilizando a integração entre frontend e backend.
- **Configuração de Políticas de CORS (Cross-Origin Resource Sharing) Restritas:**
  - **Descrição:** Adicionar um middleware no backend para permitir requisições apenas de domínios confiáveis (o domínio oficial do frontend).
  - **Justificativa:** No estado atual, o backend pode estar vulnerável a requisições de origens maliciosas. Restringir o acesso via CORS aumenta a segurança da camada de dados, garantindo que apenas a aplicação oficial possa interagir com as funções de agendamento e login.

2. Melhoria Geral (Uso e Experiência)

- **Implementação de Temas Dinâmicos (Dark/Light Mode):**
  - **Descrição:** Adicionar um seletor na interface que permita ao usuário alternar entre o tema escuro (atual) e um modo claro (Light Mode).
  - **Justificativa:** A melhoria foca na **acessibilidade e conforto visual**. Usuários que utilizam o sistema em ambientes muito iluminados podem ter dificuldade de leitura em interfaces puramente escuras. Oferecer essa flexibilidade melhora a experiência de uso e torna a plataforma mais inclusiva e profissional.

PASSO 11 – Revisão final
Tempo sugerido: 10 minutos

Conferir se o grupo possui:

1. descrição do sistema
2. escopo dos testes
3. 5 casos de teste
4. execução registrada
5. evidências
6. testes com erro
7. testes de API (se houver)
8. 2 testes automatizados
9. 2 bugs

10. melhorias

---

5. Entregáveis (Obrigatório)

Cada grupo deverá enviar no AVA:

1. **Documento com todas as etapas, com evidências (prints).**
2. **Scripts usados em PYTHON(.py)**

---

6. Critérios de Avaliação (AC3 – 1,0 ponto)

- organização e clareza do material
- execução real dos testes
- qualidade das evidências
- identificação de falhas
- coerência das melhorias propostas

---

7. Observações Finais

- a atividade deve ser realizada no próprio projeto do grupo
- o foco é testar, não apenas descrever
- encontrar falhas faz parte do trabalho de QA
- projetos sem evidências terão redução na nota
- todos os integrantes devem participar